SIMATS ENGINEERING
ASSIGNMENT 1 – CO1

COURSE NAME: DATABASE MANAGEMENT SYSTEMS
COURSE CODE: CSA05
NAME / REG NO: PRASANNA KUMAR.S [1924260253]
NATURE: Individual | Duration: 1 Week | Total Marks: 50

CO1: Analyze DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships.

Assignment title: Design, Analyse and Validate a Storage and Indexing Strategy for the Museum Condition-Report Database (Meridian Museum – Collection Loans and Exhibition Planning)

Scenario in brief

Meridian Museum borrows artifacts for temporary exhibitions. The same artifact has different catalogue descriptions in the loan register and the gallery plan, loan expiry dates are not linked to display slots, and condition reports are filed by lender name instead of by physical object. The design must separate **legal ownership, temporary custody, exhibition placement and condition evidence**. The museum has 8 collection units, 35 curators, 1,200 lenders and 9,000 cataloged artifacts; a loan belongs to one lender and contains several artifacts; display slots are numbered within an exhibition; an artifact can have successive loans; and a curator may lend a privately owned object.

Execution note (read before the results)

All SQL in Tasks 8, 9 and 12 was **actually executed** on **SQLite 3.45.1** on 2026-11-03 using Python's `sqlite3` module (foreign keys switched ON). **A MySQL 8.0 server could not be installed in the work environment** (package downloads are blocked), so no result in this report comes from MySQL. The submission DDL in Task 8 is written in MySQL 8.0 syntax; the executed version differs only in dialect (`AUTOINCREMENT` instead of `AUTO_INCREMENT`, no `CREATE DATABASE`/`USE`), and Q4 uses `julianday()` instead of `DATEDIFF()`. Error messages and timings are therefore SQLite's; MySQL error numbers shown are the documented codes for the same class of error and are labelled as not executed. The complete runner (`run_meridian.py`, seed 4211) is reproduced in Appendix A so that the run can be repeated on MySQL.

Signature of the student:

TASK 1 – Analyze the Requirements and Propose Alternative Conceptual Designs (5 Marks)

Construct three alternative conceptual approaches to loan coordination (custody periods and condition evidence), distinguishing legal ownership, loan custody and exhibition placement; show why renaming LOAN would not change the model.

Answer:

1.1 Operational users and shared information

User	Records created / updated	Information shared
Registrar	LOAN, LOAN_OBJECT, return details	Lender, loan period, artifact identifiers, due/return dates
Curator	EXHIBITION, DISPLAY_SLOT, PLACEMENT	Exhibition, slot number, artifact and display interval
Condition Officer / Conservator	CONDITION_REPORT, observations	Artifact, LoanID, inspection time, observed state and evidence
Collection Manager	ARTIFACT, classification	Permanent artifact identity, catalogue data, collection unit

The Registrar creates and closes LOAN records and verifies the lender, agreement number, start, due and return dates. The Curator supplies exhibition and placement details; the Conservator records the physical condition at dispatch, during display and at return. All users share a stable **ArtifactID** and **LoanID**, so custody, placement and condition evidence refer to the same physical object and the same event.

1.2 Alternative A – dated relationship (identity carried by the relationship)

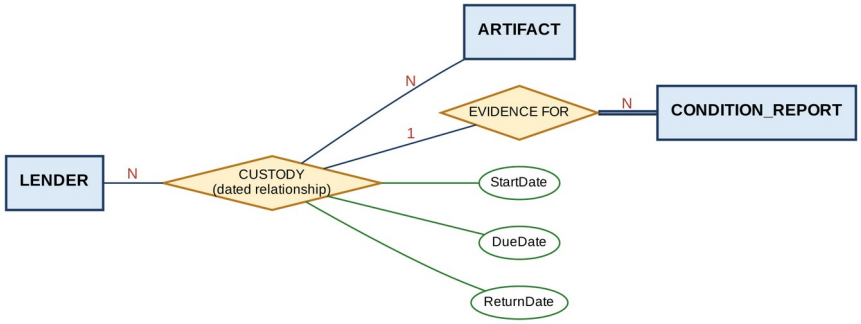


Figure 1 – Alternative A: LENDER and ARTIFACT joined by a dated CUSTODY relationship

LENDER and ARTIFACT are connected through a dated CUSTODY relationship carrying StartDate, DueDate and ReturnDate; condition reports are attached to that relationship. The relationship is identified by its participants plus the start date (or by a generated key in the relational design). **Strength:** repeated borrowing is represented when the date is part of the identity. **Limitation:** other records cannot easily reference the relationship, and one loan with many artifacts is not a single operational event – the agreement would be repeated for each artifact. **Failure case resolved:** a return report for an artifact from one custody period must not be mistaken for evidence from a later loan by the same lender.

1.3 Alternative B – independently identified LOAN occurrence (preferred original design)

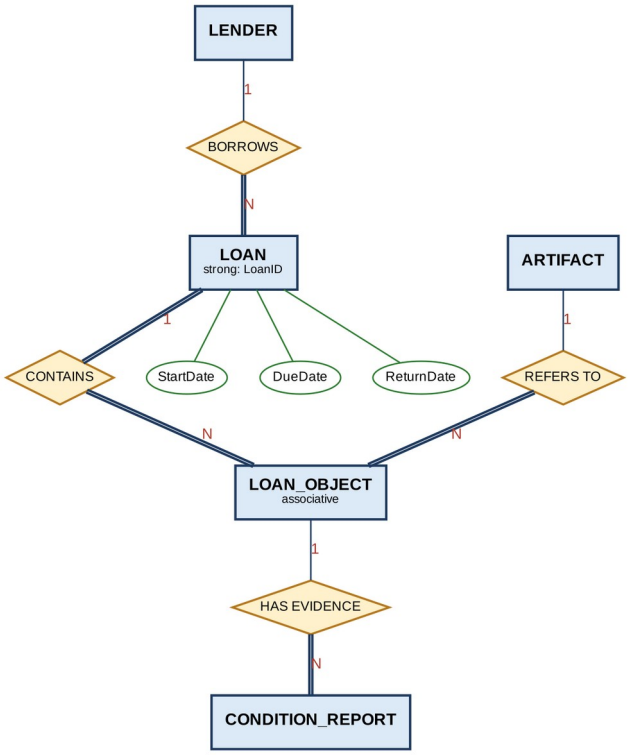


Figure 2 – Alternative B: LOAN as a strong entity; LOAN_OBJECT as associative entity

LOAN is a strong entity with LoanID; LENDER 1:N LOAN records the responsible lender; LOAN_OBJECT (associative entity) records the artifact lines of a loan; CONDITION_REPORT refers to the artifact and to the specific LOAN_OBJECT line; exhibition placement is a separate fact and cannot change custody. **Strength:** one LoanID identifies one custody event even when it holds several artifacts, and the same artifact can have successive LOAN_OBJECT rows. **Failure case resolved:** two loans from the same lender for the same artifact at different times must not overwrite each other – a stable LoanID preserves both custody periods and their evidence.

1.4 Alternative C – shared PERSON identity with role subtypes

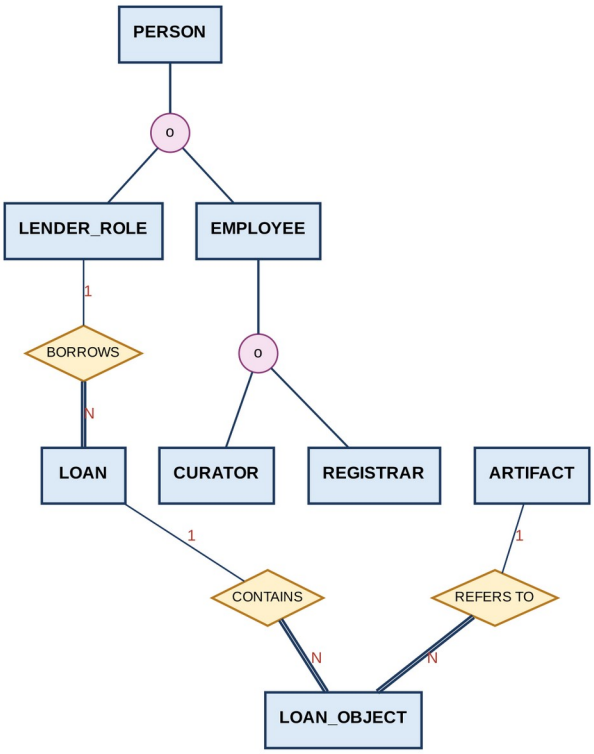


Figure 3 – Alternative C: PERSON generalised into LENDER_ROLE and EMPLOYEE (CURATOR, REGISTRAR); LOAN structure retained

This alternative changes **identity** rather than event identity: one PERSON entity with overlapping roles LENDER_ROLE and EMPLOYEE (specialised into CURATOR and REGISTRAR). The LOAN structure of B is kept, but lender and employee roles point to the same person. **Strength:** when a curator lends a privately owned object, one PERSON row carries both roles, contact data is not duplicated and condition evidence stays linked to the artifact and the custody event. **Limitation:** role/subtype rules need careful constraints and extra joins.

1.5 Comparison, selection and why renaming LOAN changes nothing

Design	Main modelling distinction	What it solves	Main weakness
A – dated relationship	Dates are attributes of the LENDER–ARTIFACT relationship	Repeated custody periods separated by date	Multi-artifact loan not one shared event
B – LOAN entity	Every loan independently identified by LoanID	One agreement, several artifacts, successive loans, traceable evidence	Needs LOAN_OBJECT and extra joins
C – PERSON roles	One person identity holds several roles	No duplicate identity when a curator is also a lender	Role constraints and subtype joins

Selected design: B combined with C – an independently identified LOAN with shared PERSON identity and role-based participation. Renaming LOAN (for example to CUSTODY or AGREEMENT) would leave the model unchanged because the distinction between A and B is what carries the identity – a relationship identified by its participants and date, or an entity identified by its own key – and the degree of the association (binary lender–artifact versus lender–loan–line–artifact). A new name does not change identity, participation or history. **Counterexample from the case:** if the same lender borrows the same artifact twice, A can separate the two only through the date, while B uses two LoanIDs; and a loan for five artifacts needs five dated custody rows in A but one LOAN with five lines in B. Ownership is deliberately **not** treated as custody; Task 12 adds a separate ownership history.

TASK 2 – Identify Entities, Attributes and Keys (2 Marks)

Build an entity and key catalog, distinguishing real-world identity from record labels; decide whether an exhibition name can identify a display slot.

Answer:

Entity	Important attributes	Primary key / candidate key
PERSON	PersonID, GivenName, MiddleName, FamilyName, Email, Address	PK PersonID; Email candidate only if verified unique
LENDER_ROLE	PersonID, LenderSince, LenderStatus	PK/FK PersonID
EMPLOYEE	PersonID, EmployeeNo, HireDate, Role	PK/FK PersonID; candidate EmployeeNo
CURATOR / REGISTRAR	PersonID, Specialisation / AuthorisationLevel	PK/FK PersonID
ARTIFACT	ArtifactID, AccessionNo, Title, CollectionUnitID, CollectionTypeID	PK ArtifactID; candidate AccessionNo
LOAN	LoanID, LenderPersonID, AgreementNo, StartDate, DueDate, ReturnDate	PK LoanID; candidate AgreementNo (by policy)
LOAN_OBJECT	LoanID, ArtifactID, LineNo, LoanValue, DispatchCondition	PK (LoanID, ArtifactID); alternate (LoanID, LineNo)
EXHIBITION	ExhibitionID, Name, StartDate, EndDate	PK ExhibitionID; Name is not reliable
DISPLAY_SLOT (weak)	ExhibitionID, SlotNo, LocationLabel	PK (ExhibitionID, SlotNo)
PLACEMENT	PlacementID, ArtifactID, ExhibitionID, SlotNo, FromDate, ToDate	PK PlacementID; FK (ExhibitionID, SlotNo)
CONDITION_REPORT	ReportID, LoanID, ArtifactID, InspectedAt, ReportType, Notes	PK ReportID; FK (LoanID, ArtifactID)
COLLECTION_UNIT / TYPE	CollectionUnitID, UnitName / CollectionTypeID, TypeName, Description	PK surrogate id; UnitName, TypeName unique
PERSON_PHONE	PersonID, PhoneNumber, PhoneType	PK (PersonID, PhoneNumber)

2.1 Entity identity and the weak entity

PERSON, ARTIFACT, LOAN, EXHIBITION, CONDITION_REPORT and COLLECTION_UNIT have independent identities. **DISPLAY_SLOT** is existence-dependent on EXHIBITION: SlotNo 4 in Exhibition A is not the same slot as SlotNo 4 in Exhibition B. It is a weak entity with partial key SlotNo and identifying owner ExhibitionID; its full key is **(ExhibitionID, SlotNo)**.

2.2 Attribute classification

Attribute	Classification	Reason
Contact information (phone)	Multivalued	A person may have several phone numbers → PERSON_PHONE
Person name	Composite	Given, middle and family names stored separately
CollectionType	Coded / reference	Separate entity because it has its own description
StartDate, DueDate, ReturnDate	Stored simple (date)	Actual operational facts
Loan duration, overdue days	Derived	Calculated from the dates and the current date
Condition summary	Derived	Computed from the stored condition reports

2.3 Keys and why a name is not an identifier

An exhibition name cannot identify a display slot: names repeat, change, vary in spelling and can be reused in different years, and a slot number is unique only inside its exhibition, so the owner-scoped key is (ExhibitionID, SlotNo). Stable surrogate keys (PersonID, ArtifactID, LoanID) avoid dependence on names; business candidate keys (AccessionNo, EmployeeNo, AgreementNo) get UNIQUE constraints. **Concurrent registration:** surrogate keys are issued by the database (`AUTO_INCREMENT`, or a UUID) and the business key carries a UNIQUE index, so two users registering the same accession number at the same time cannot

both succeed – the unique index lets one insert commit and rejects the other. This was demonstrated in Task 8 (duplicate `AccessionNo` rejected).

TASK 3 – Determine Relationships and Structural Constraints (6 Marks)

Derive the relationship constraints connecting LENDER, operational events and dependent evidence; explain the composite placement-to-slot constraint.

Answer:

3.1 Binary relationships and a possible ternary relationship

The core associations are binary: LENDER_ROLE–LOAN (a lender has many loans; every loan has exactly one responsible lender), LOAN–LOAN_OBJECT, ARTIFACT–LOAN_OBJECT, EXHIBITION–DISPLAY_SLOT and ARTIFACT–PLACEMENT. A possible ternary association is LENDER–CURATOR–EXHIBITION (a curator coordinates a particular lender's contribution to a particular exhibition). It is justified only if the combination itself has meaning and cannot be rebuilt from binary links; otherwise the binary relationships are used.

3.2 Participation and cardinality (min–max)

Relationship	Participation	Explanation
LENDER_ROLE – LOAN	LENDER_ROLE 0..N; LOAN exactly 1	A registered lender may never borrow; every loan has one responsible lender
LOAN – LOAN_OBJECT	LOAN 1..N lines; LOAN_OBJECT exactly 1 loan	A valid loan has at least one artifact line; each line belongs to one loan
ARTIFACT – LOAN_OBJECT	ARTIFACT 0..N; LOAN_OBJECT exactly 1 artifact	An artifact may never have been loaned; each line names one artifact
EXHIBITION – DISPLAY_SLOT	EXHIBITION 0..N; DISPLAY_SLOT exactly 1	An exhibition may exist before slots are defined; a slot cannot exist without its exhibition
ARTIFACT – CONDITION_REPORT	ARTIFACT 0..N; REPORT exactly 1 artifact	An artifact may lack a report (queryable); every report concerns one artifact

Missing LOAN_OBJECT evidence is not a harmless blank: a loan with no lines has no documented artifacts and should be rejected before confirmation. A simple row CHECK cannot count child rows; this was confirmed in Task 8, where a loan without lines was **accepted** by the schema and had to be found by a detection query.

3.3 Where dated or quantitative facts belong

- **LOAN:** agreement number, lender, custody start, due date, actual return date, registrar.
- **LOAN_OBJECT:** artifact, line number, line valuation (LoanValue), dispatch condition.
- **PLACEMENT:** exhibition, slot, artifact and from/to dates (an artifact can be moved between slots).
- **CONDITION_REPORT:** inspection time, inspector, report type, notes, evidence reference – identifies the artifact and, for loan inspections, the custody line.
- **Ownership shares and effective dates:** the ownership history (Task 12), never LOAN or LOAN_OBJECT.

3.4 Ternary instance and binary-projection test

COORDINATION(Lender, Curator, Exhibition) = {(L1,C1,E1), (L1,C2,E2), (L2,C1,E2)}. Projections: Lender–Curator = {(L1,C1),(L1,C2),(L2,C1)}; Lender–Exhibition = {(L1,E1),(L1,E2),(L2,E2)}; Curator–Exhibition = {(C1,E1),(C2,E2),(C1,E2)}. Joining the three projections produces the extra tuple **(L1,C1,E2)**, which is not in the original relation, so the binary projections do **not** reconstruct the ternary fact. A real three-way fact therefore needs a ternary associative entity keyed on all three participants (or a surrogate key plus UNIQUE); independent assignments use binary relationships. For custody evidence the same risk arises if a report is linked separately to a lender and to a curator and then joined to an exhibition – it would imply a loan/exhibition combination that never occurred. Linking the report directly to the LOAN_OBJECT line avoids this.

TASK 4 – Construct the ER Diagram (4 Marks)

Produce a Chen ER representation of the preferred model reconciled with the entity catalog, with separate paths for borrowed objects and condition reports.

Answer:

The diagram is split into two pages to stay readable. **Repeated entity references:** ARTIFACT appears in Figure 4 and again in Figure 5 (and LOAN_OBJECT in Figure 5) as a drawing device only – each is one entity in the schema. Figure 4 carries the custody path (ARTIFACT → LOAN_OBJECT → LOAN) and the condition-evidence path (LOAN_OBJECT → CONDITION_REPORT); Figure 5 carries classification and exhibition placement.

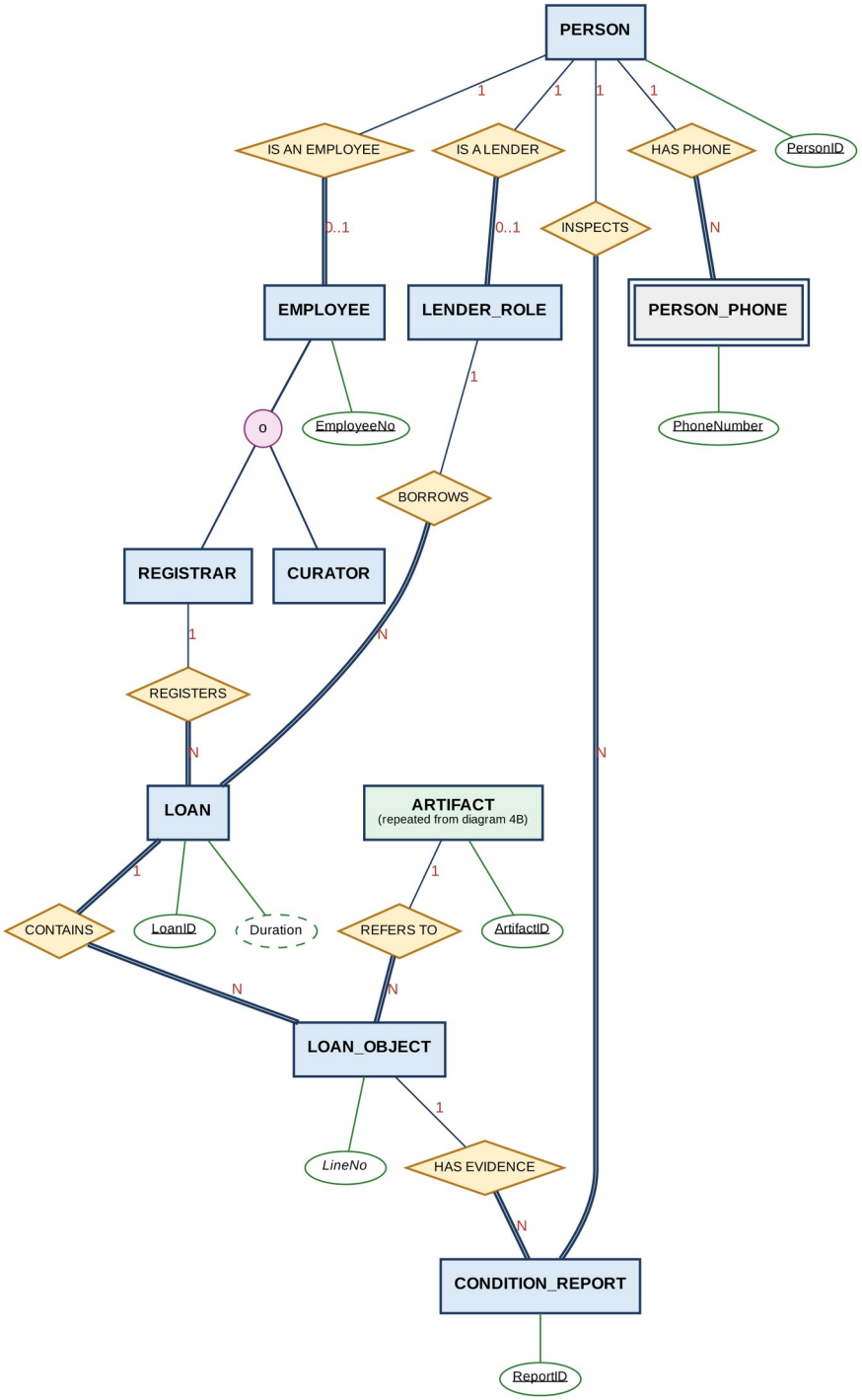


Figure 4 – Chen ER (part 1): people, custody and condition evidence

Legend: rectangle = entity; double rectangle = weak entity; diamond = relationship; double diamond = identifying relationship; ellipse = attribute (underlined = primary key, italic = partial key, dashed = derived); double line = total (compulsory) participation; circle ‘o’ = overlapping specialisation; labels 1 / N = maximum cardinality, 0..1 = optional one.

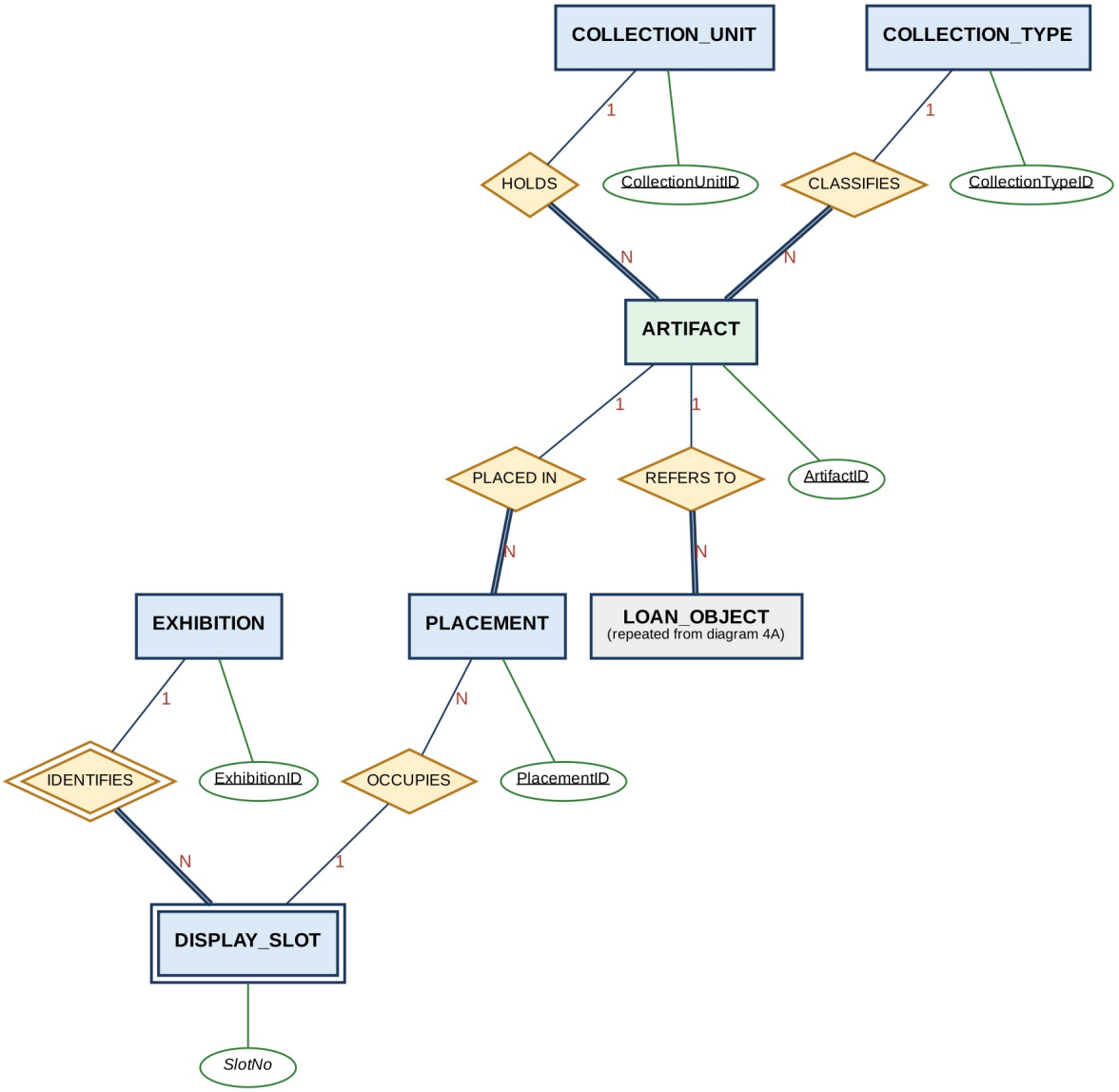


Figure 5 – Chen ER (part 2): artifact classification and exhibition placement

Owner-scoped key: DISPLAY_SLOT is identified by EXHIBITION through the identifying relationship IDENTIFIES, so its key is (ExhibitionID, SlotNo). **Composite placement-to-slot constraint:** PLACEMENT must reference the pair (ExhibitionID, SlotNo) as one composite foreign key; SlotNo alone is invalid because slot 2 exists in several exhibitions. In Task 8 an insert into slot 7 of exhibition 1 (which does not exist) was rejected. Reconciliation with Task 2/3: LENDER_ROLE and EMPLOYEE are shown 0..1 per person because their primary key is the PersonID, exactly as in the schema.

4.1 Attribute catalog (PK / FK*)

Entity	Attributes
PERSON	PersonID (PK), GivenName, MiddleName, FamilyName, Email, Address
PERSON_PHONE	PersonID (PK, FK*), PhoneNumber (PK), PhoneType
LENDER_ROLE	PersonID (PK, FK*), LenderSince, LenderStatus
EMPLOYEE	PersonID (PK, FK*), EmployeeNo (UQ), HireDate, Role
CURATOR / REGISTRAR	PersonID (PK, FK* → EMPLOYEE), Specialisation / AuthorisationLevel
ARTIFACT	ArtifactID (PK), AccessionNo (UQ), Title, Description, CollectionUnitID*, CollectionTypeID*, ArtifactValue
LOAN	LoanID (PK), LenderPersonID*, AgreementNo (UQ), StartDate, DueDate, ReturnDate, RegistrarPersonID*
LOAN_OBJECT	LoanID (PK, FK*), ArtifactID (PK, FK*), LineNo (UQ with LoanID), LoanValue, DispatchCondition
CONDITION_REPORT	ReportID (PK), LoanID*, ArtifactID* (composite FK to LOAN_OBJECT), InspectorPersonID*, InspectedAt, ReportType, Notes, EvidenceURI
EXHIBITION	ExhibitionID (PK), Name, StartDate, EndDate
DISPLAY_SLOT	ExhibitionID (PK, FK*), SlotNo (PK), LocationLabel
PLACEMENT	PlacementID (PK), ArtifactID*, ExhibitionID*, SlotNo* (composite FK to DISPLAY_SLOT), FromDate, ToDate
COLLECTION_UNIT / TYPE	CollectionUnitID (PK), UnitName (UQ) / CollectionTypeID (PK), TypeName (UQ), Description

TASK 5 – Extend the Model to an EER Model (3 Marks)

Use EER concepts to resolve overlapping roles and mixed contact identities; handle a curator lending a personal collection.

Answer:

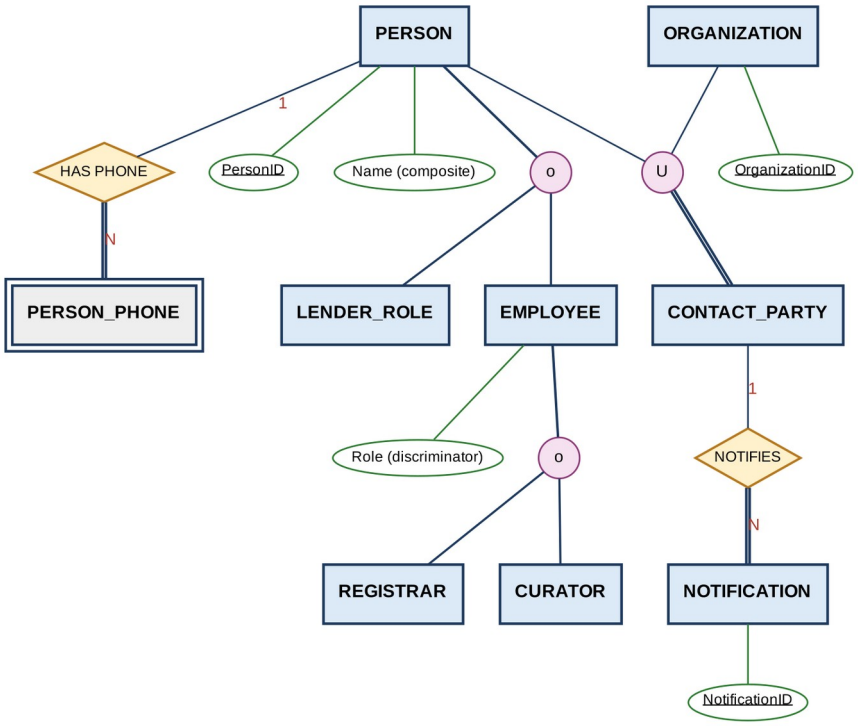


Figure 6 – EER: generalisation, specialisation and the CONTACT_PARTY union category

5.1 Generalisation and specialisation

PERSON is the supertype holding identity and contact data. LENDER_ROLE and EMPLOYEE are **overlapping** roles of a person (a curator can also lend), and EMPLOYEE is specialised into CURATOR and REGISTRAR. The employee specialisation is **partial**: the case names curators and registrars but does not

state they are the only employee types (Role also allows OTHER). The `Role` discriminator controls subtype membership (Role='CURATOR' permits a CURATOR row); if one employee may hold several employee roles, a single-valued discriminator is insufficient and an EMPLOYEE_ROLE association table is used. A curator who lends a private object has the same PersonID in EMPLOYEE/CURATOR and in LENDER_ROLE; the loan is still a separate custody occurrence and its condition reports stay linked to the loan line.

5.2 Union category for notification contact

A notification contact may be a person or an organisation, so CONTACT_PARTY is a **category (union) of PERSON and ORGANIZATION**; a NOTIFICATION refers to exactly one CONTACT_PARTY. Relationally this is a common ContactParty table with a type discriminator, or two nullable foreign keys with a CHECK that exactly one is non-NULL (this pattern is executed for ARTIFACT_OWNER in Task 12).

5.3 Subtypes versus one wide record

A single wide table with lender, curator and registrar columns would hold many NULLs, mix identity with role and duplicate a person who has several roles. The subtype design stores common details once in PERSON and role details only where they apply, gives clearer referential integrity and supports the curator-lender case.

TASK 6 – Model Weak Entities and Complex Attributes (3 Marks)

Resolve owner-dependent identities and complex attributes; distinguish an artifact value from a loan-line valuation.

Answer:

6.1 DISPLAY_SLOT identity

DISPLAY_SLOT is weak: no independent identity outside its exhibition. Owner = EXHIBITION, partial key = SlotNo, primary key = (ExhibitionID, SlotNo); the identifying relationship is mandatory. An ordinary foreign key does not make an entity weak – the identifying relationship and the owner-dependent key do.

6.2 Multivalued and composite attributes

Telephone numbers are not stored as comma-separated text: PERSON_PHONE has one row per number with key (PersonID, PhoneNumber), so a number can be searched, validated, updated or deleted individually. PersonName is composite and stored as GivenName, MiddleName, FamilyName; the display name is derived.

6.3 Derived attributes and coded concepts

Attribute / concept	Storage decision	Reason
Loan duration	Derived from StartDate and ReturnDate (or current date)	Avoids stale values (Q4 computes it)
Overdue days	Derived from DueDate and current date for open loans	Changes with time (Q1 finds overdue loans)
Current custody status	Derived from ReturnDate and date rules	Must agree with the custody dates
Latest condition summary	Derived from the latest condition report	Evidence stays at report level (Q5)
Condition history	Store every report	Historical evidence must not be overwritten
CollectionType	Separate reference entity	Has stable code, label and description
ReportType	Reference values (CHECK list here; entity if descriptions are maintained)	Avoids inconsistent free text; invalid value rejected in Task 8

ArtifactValue vs LoanValue: ArtifactValue is the museum's estimated or insured value of the object; LoanValue is the value recorded for one loan line and may reflect the agreement or insurance valuation for that loan. A later change to ArtifactValue must not rewrite a historical LoanValue; if values change over time, dated valuation records are kept. Condition summaries such as ‘good’ or ‘minor crack’ are computed from timestamped reports, and the source evidence stays immutable so that later assessments cannot erase the condition at dispatch or return.

TASK 7 – Map the EER Model to a Relational Schema (4 Marks)

Derive relations from the preferred EER model and explain the mapping decisions; map several artifacts within one loan.

Answer:

Mapping rules: each strong entity becomes a relation; the weak entity includes its owner's key; a 1:N relationship places the 1-side key as a foreign key on the N-side; an M:N relationship (LOAN–ARTIFACT) becomes the associative relation LOAN_OBJECT; multivalued attributes become separate relations; and the overlapping specialisation uses **a supertype table plus one table per subtype** (each subtype's PersonID is both PK and FK).

7.1 Relational schema catalog

```
PERSON(PersonID PK, GivenName, MiddleName, FamilyName, Email, Address)
PERSON_PHONE(PersonID PK/FK -> PERSON, PhoneNumber PK, PhoneType)
LENDER_ROLE(PersonID PK/FK -> PERSON, LenderSince, LenderStatus)
EMPLOYEE(PersonID PK/FK -> PERSON, EmployeeNo UNIQUE, HireDate, Role)
CURATOR(PersonID PK/FK -> EMPLOYEE, Specialisation)
REGISTRAR(PersonID PK/FK -> EMPLOYEE, AuthorisationLevel)
COLLECTION_UNIT(CollectionUnitID PK, UnitName UNIQUE)
COLLECTION_TYPE(CollectionTypeID PK, TypeName UNIQUE, Description)
ARTIFACT(ArtifactID PK, AccessionNo UNIQUE, Title, Description,
         CollectionUnitID FK, CollectionTypeID FK, ArtifactValue)
LOAN(LoanID PK, LenderPersonID FK -> LENDER_ROLE, AgreementNo UNIQUE,
     StartDate, DueDate, ReturnDate, RegistrarPersonID FK -> REGISTRAR)
LOAN_OBJECT(LoanID PK/FK -> LOAN, ArtifactID PK/FK -> ARTIFACT, LineNo, LoanValue,
            DispatchCondition, UNIQUE(LoanID, LineNo))
EXHIBITION(ExhibitionID PK, Name, StartDate, EndDate)
DISPLAY_SLOT(ExhibitionID PK/FK -> EXHIBITION, SlotNo PK, LocationLabel)
PLACEMENT(PlacementID PK, ArtifactID FK, ExhibitionID, SlotNo, FromDate, ToDate,
          FK(ExhibitionID, SlotNo) -> DISPLAY_SLOT)
CONDITION_REPORT(ReportID PK, LoanID, ArtifactID, InspectorPersonID FK -> PERSON,
          InspectedAt, ReportType, Notes, EvidenceURI,
          FK(LoanID, ArtifactID) -> LOAN_OBJECT)
```

7.2 Mapping decisions and deletion behaviour

- LOAN_OBJECT maps the M:N relationship: one loan holds several artifacts and an artifact appears in several historical loans.
- DISPLAY_SLOT carries the identifying owner key; PLACEMENT references the composite pair.
- CONDITION_REPORT has a composite FK to LOAN_OBJECT so a report tied to a loan cannot refer to an artifact that was not on that loan (tested in Task 8).
- Historical LOAN and LOAN_OBJECT rows are retained: **RESTRICT** on deleting a lender, artifact or loan that is referenced (tested: deletes were rejected); use status flags for operational retirement; never cascade-delete loan history or condition evidence.
- Indexes (created in the executed run): LOAN(LenderPersonID, StartDate), LOAN(DueDate, ReturnDate), LOAN_OBJECT(ArtifactID), CONDITION_REPORT(ArtifactID, InspectedAt), CONDITION_REPORT(LoanID, ArtifactID), PLACEMENT(ArtifactID, FromDate, ToDate), PLACEMENT(ExhibitionID, SlotNo), ARTIFACT(CollectionUnitID).

TASK 8 – Implement the Schema and Populate Instances (4 Marks)

Implement the relations in MySQL, build reproducible valid states for 100, 1,000, 10,000 and 100,000 lenders (seed 4211), attempt invalid writes and keep the actual failure messages.

Answer:

8.1 DDL (MySQL 8.0 syntax – submission version)

```
CREATE DATABASE IF NOT EXISTS meridian_museum;
USE meridian_museum;

CREATE TABLE Person (
  PersonID BIGINT PRIMARY KEY AUTO_INCREMENT,
  GivenName VARCHAR(60) NOT NULL,
  MiddleName VARCHAR(60),
  FamilyName VARCHAR(60) NOT NULL,
  Email VARCHAR(150) UNIQUE,
  Address VARCHAR(255)
);

CREATE TABLE PersonPhone (
  PersonID BIGINT NOT NULL,
  PhoneNumber VARCHAR(25) NOT NULL,
  PhoneType VARCHAR(20) NOT NULL,
  PRIMARY KEY (PersonID, PhoneNumber),
  FOREIGN KEY (PersonID) REFERENCES Person(PersonID)
    ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE LenderRole (
  PersonID BIGINT PRIMARY KEY,
  LenderSince DATE,
  LenderStatus VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
  FOREIGN KEY (PersonID) REFERENCES Person(PersonID)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CHECK (LenderStatus IN ('ACTIVE','INACTIVE'))
);

CREATE TABLE Employee (
  PersonID BIGINT PRIMARY KEY,
  EmployeeNo VARCHAR(30) NOT NULL UNIQUE,
  HireDate DATE NOT NULL,
  Role VARCHAR(20) NOT NULL,
  FOREIGN KEY (PersonID) REFERENCES Person(PersonID)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CHECK (Role IN ('CURATOR','REGISTRAR','OTHER'))
```

```
);

CREATE TABLE Curator (
  PersonID BIGINT PRIMARY KEY,
  Specialisation VARCHAR(100),
  FOREIGN KEY (PersonID) REFERENCES Employee(PersonID)
    ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE Registrar (
  PersonID BIGINT PRIMARY KEY,
  AuthorisationLevel VARCHAR(30),
  FOREIGN KEY (PersonID) REFERENCES Employee(PersonID)
    ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE CollectionUnit (
  CollectionUnitID INT PRIMARY KEY AUTO_INCREMENT,
  UnitName VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE CollectionType (
  CollectionTypeID INT PRIMARY KEY AUTO_INCREMENT,
  TypeName VARCHAR(80) NOT NULL UNIQUE,
  Description VARCHAR(255)
);

CREATE TABLE Artifact (
  ArtifactID BIGINT PRIMARY KEY AUTO_INCREMENT,
  AccessionNo VARCHAR(40) NOT NULL UNIQUE,
  Title VARCHAR(180) NOT NULL,
  Description TEXT,
  CollectionUnitID INT NOT NULL,
  CollectionTypeID INT NOT NULL,
  ArtifactValue DECIMAL(14,2),
  FOREIGN KEY (CollectionUnitID) REFERENCES CollectionUnit(CollectionUnitID),
  FOREIGN KEY (CollectionTypeID) REFERENCES CollectionType(CollectionTypeID),
  CHECK (ArtifactValue IS NULL OR ArtifactValue >= 0)
);

CREATE TABLE Loan (
  LoanID BIGINT PRIMARY KEY AUTO_INCREMENT,
  LenderPersonID BIGINT NOT NULL,
  AgreementNo VARCHAR(50) NOT NULL UNIQUE,
  StartDate DATE NOT NULL,
  DueDate DATE NOT NULL,
  ReturnDate DATE,
  RegistrarPersonID BIGINT NOT NULL,
  FOREIGN KEY (LenderPersonID) REFERENCES LenderRole(PersonID),
  FOREIGN KEY (RegistrarPersonID) REFERENCES Registrar(PersonID),
  CHECK (DueDate >= StartDate),
  CHECK (ReturnDate IS NULL OR ReturnDate >= StartDate)
);

CREATE TABLE LoanObject (
  LoanID BIGINT NOT NULL,
  ArtifactID BIGINT NOT NULL,
  LineNo INT NOT NULL,
  LoanValue DECIMAL(14,2),
  DispatchCondition VARCHAR(100),
  PRIMARY KEY (LoanID, ArtifactID),
  UNIQUE (LoanID, LineNo),
  FOREIGN KEY (LoanID) REFERENCES Loan(LoanID) ON DELETE RESTRICT,
  FOREIGN KEY (ArtifactID) REFERENCES Artifact(ArtifactID) ON DELETE RESTRICT,
  CHECK (LineNo > 0),
  CHECK (LoanValue IS NULL OR LoanValue >= 0)
);

CREATE TABLE Exhibition (
  ExhibitionID INT PRIMARY KEY AUTO_INCREMENT,
  Name VARCHAR(150) NOT NULL,
  StartDate DATE NOT NULL,
  EndDate DATE NOT NULL,
  CHECK (EndDate >= StartDate)
);
```

```
CREATE TABLE DisplaySlot (
  ExhibitionID INT NOT NULL,
  SlotNo INT NOT NULL,
  LocationLabel VARCHAR(100),
  PRIMARY KEY (ExhibitionID, SlotNo),
  FOREIGN KEY (ExhibitionID) REFERENCES Exhibition(ExhibitionID)
    ON DELETE RESTRICT,
  CHECK (SlotNo > 0)
);

CREATE TABLE Placement (
  PlacementID BIGINT PRIMARY KEY AUTO_INCREMENT,
  ArtifactID BIGINT NOT NULL,
  ExhibitionID INT NOT NULL,
  SlotNo INT NOT NULL,
  FromDate DATE NOT NULL,
  ToDate DATE,
  FOREIGN KEY (ArtifactID) REFERENCES Artifact(ArtifactID),
  FOREIGN KEY (ExhibitionID, SlotNo)
    REFERENCES DisplaySlot(ExhibitionID, SlotNo),
  CHECK (ToDate IS NULL OR ToDate >= FromDate)
);

CREATE TABLE ConditionReport (
  ReportID BIGINT PRIMARY KEY AUTO_INCREMENT,
  LoanID BIGINT NOT NULL,
  ArtifactID BIGINT NOT NULL,
  InspectorPersonID BIGINT NOT NULL,
  InspectedAt DATETIME NOT NULL,
  ReportType VARCHAR(30) NOT NULL,
  Notes TEXT,
  EvidenceURI VARCHAR(500),
  FOREIGN KEY (LoanID, ArtifactID)
    REFERENCES LoanObject(LoanID, ArtifactID) ON DELETE RESTRICT,
  FOREIGN KEY (InspectorPersonID) REFERENCES Person(PersonID),
  CHECK (ReportType IN ('DISPATCH','IN_DISPLAY','RETURN','OTHER'))
);

CREATE INDEX ix_loan_lender  ON Loan(LenderPersonID, StartDate);
CREATE INDEX ix_loan_due     ON Loan(DueDate, ReturnDate);
CREATE INDEX ix_lo_artifact  ON LoanObject(ArtifactID);
CREATE INDEX ix_cr_artifact  ON ConditionReport(ArtifactID, InspectedAt);
CREATE INDEX ix_cr_loanobj   ON ConditionReport(LoanID, ArtifactID);
CREATE INDEX ix_pl_artifact  ON Placement(ArtifactID, FromDate, ToDate);
CREATE INDEX ix_pl_slot      ON Placement(ExhibitionID, SlotNo);
CREATE INDEX ix_art_unit     ON Artifact(CollectionUnitID);
```

8.2 What was actually executed

The same tables, keys, CHECKs and indexes were executed on SQLite 3.45.1 (`run/schema_sqlite.sql`, dialect changes only). The starter sample of the assignment was loaded first (2 artifacts, 1 loan with 2 lines, 2 dispatch reports); all counts below are read back from the database.

Table	Rows	Table	Rows
CollectionUnit	8	Registrar	1
CollectionType	4	Artifact	2
Person	3	Loan	1
LenderRole	2	LoanObject	2
Employee	1	ConditionReport	2

8.3 Reproducible generation with seed 4211

`random.Random(4211)` is created once per population. **Linked-row rule:** parents are inserted first and every foreign key is drawn from keys that already exist – LenderPersonID from the generated lender list, RegistrarPersonID from registrars, the two ArtifactIDs of a loan by `rng.sample(range(1, n_artifacts+1), 2)` (so a loan never lists one artifact twice), inspectors from curators, placement slots from the 12 × 20 generated slots. **Proportions:** artifacts = 7.5 × lenders (the case ratio 9,000 : 1,200), loans = lenders / 10, two lines per loan, curators ≈ 35 per 1,200 lenders, registrars = lenders / 100, 12 exhibitions × 20 slots. 70 % of loans are returned (ReturnDate within ±10 days of DueDate, never before StartDate); every line gets a DISPATCH report, 90 % of returned lines a RETURN report, 20 % an IN_DISPLAY report, and 50 % of lines a placement. Foreign keys were ON during loading and `PRAGMA foreign_key_check` returned **0 violations** for every size.

Lenders	Persons	Staff	Artifacts	Loans	Lines	Reports	Placed	Load s
100	105	5	750	10	20	34	10	0.03
1000	1039	39	7500	100	200	382	98	0.06
10000	10392	392	75000	1000	2000	3658	982	0.38
100000	103917	3917	750000	10000	20000	36490	9988	3.73

8.4 Invalid writes and the actual failure messages

Sixteen invalid statements were run against the starter sample (foreign keys ON). Every one was rejected and left the data unchanged. The ‘MySQL 8.0’ column gives the documented error number for that kind of violation; it is **not** an executed result.

#	Invalid write (class)	Executed statement	Actual SQLite message	MySQL 8.0 (not run)
1	Identity - duplicate accession number (UNIQUE)	INSERT INTO Artifact(AccessionNo, Title, CollectionUnitID, CollectionTypeID) VALUES ('MM-0001','Duplicate test',1,1)	UNIQUE constraint failed: Artifact.AccessionNo	1062 duplicate entry
2	Identity - duplicate lender e-mail (UNIQUE)	INSERT INTO Person(GivenName, FamilyName, Email) VALUES ('Dup','Person','asha@example.test')	UNIQUE constraint failed: Person.Email	1062 duplicate entry
3	Identity - duplicate PersonID in LenderRole (PRIMARY KEY)	INSERT INTO LenderRole(PersonID, LenderSince) VALUES (1,'2025-01-01')	UNIQUE constraint failed: LenderRole.PersonID	1062 duplicate entry
4	Workflow - due date before start date (CHECK)	INSERT INTO Loan(LenderPersonID, AgreementNo, StartDate, DueDate, RegistrarPersonID) VALUES (1,'BAD-DATE','2026-05-10','2026-05-01',3)	CHECK constraint failed: DueDate >= StartDate	3819 check constraint violated
5	Workflow - return date before start date (CHECK)	INSERT INTO Loan(LenderPersonID, AgreementNo, StartDate, DueDate, ReturnDate, RegistrarPersonID) VALUES (1,'BAD-RET','2026-05-10','2026-06-01','2026-05-02',3)	CHECK constraint failed: ReturnDate IS NULL OR ReturnDate >= StartDate	3819 check constraint violated
6	Workflow - loan by a person who is not a registrar (FOREIGN KEY)	INSERT INTO Loan(LenderPersonID, AgreementNo, StartDate, DueDate, RegistrarPersonID) VALUES (1,'BAD-REG','2026-05-01','2026-06-01',2)	FOREIGN KEY constraint failed	1452 cannot add / update child row
7	Workflow - condition report for an artifact not on that loan (composite FOREIGN KEY)	INSERT INTO ConditionReport(LoanID, ArtifactID, InspectorPersonID, InspectedAt, ReportType) VALUES (1,3,3,'2026-02-01 09:00:00','RETURN')	FOREIGN KEY constraint failed	1452 cannot add / update child row
8	Category - unknown collection type (FOREIGN KEY)	INSERT INTO Artifact(AccessionNo, Title, CollectionUnitID, CollectionTypeID) VALUES ('MM-9999','Invalid type',1,9999)	FOREIGN KEY constraint failed	1452 cannot add / update child row
9	Category - invalid report type (CHECK)	INSERT INTO ConditionReport(LoanID, ArtifactID, InspectorPersonID, InspectedAt, ReportType) VALUES (1,1,3,'2026-02-01 09:00:00','BOGUS')	CHECK constraint failed: ReportType IN ('DISPATCH','IN_DISPLAY','RETURN','OTHER')	3819 check constraint violated
10	Category - invalid lender status (CHECK)	INSERT INTO LenderRole(PersonID, LenderStatus) VALUES (3,'PENDING')	CHECK constraint failed: LenderStatus IN ('ACTIVE','INACTIVE')	3819 check constraint violated
11	Category - invalid employee role (CHECK)	INSERT INTO Employee(PersonID, EmployeeNo, HireDate, Role) VALUES (2,'EMP-002','2024-01-01','JANITOR')	CHECK constraint failed: Role IN ('CURATOR','REGISTRAR','OTHER')	3819 check constraint violated
12	Valuation - negative loan valuation (CHECK)	INSERT INTO LoanObject(LoanID, ArtifactID, LineNo, LoanValue) VALUES (1,1,99,-500)	CHECK constraint failed: LoanValue IS NULL OR LoanValue >= 0	3819 check constraint violated
13	Valuation - negative artifact value (CHECK)	INSERT INTO Artifact(AccessionNo, Title, CollectionUnitID, CollectionTypeID, ArtifactValue) VALUES ('MM-0004','Negative',1,1,-1)	CHECK constraint failed: ArtifactValue IS NULL OR ArtifactValue >= 0	3819 check constraint violated
14	Slot - placement into a slot that does not exist in that exhibition (composite FOREIGN KEY)	INSERT INTO Placement(ArtifactID, ExhibitionID, SlotNo, FromDate) VALUES (2,1,7,'2026-02-01')	FOREIGN KEY constraint failed	1452 cannot add / update child row
15	History - deleting an artifact referenced by a loan line (RESTRICT)	DELETE FROM Artifact WHERE ArtifactID = 1	FOREIGN KEY constraint failed	1451 cannot delete parent row
16	History - deleting a lender who has a loan (RESTRICT)	DELETE FROM LenderRole WHERE PersonID = 1	FOREIGN KEY constraint failed	1451 cannot delete parent row

8.5 Business rules that need more than a row-level CHECK

Three cross-row rules were tried and **the schema accepted all three** – evidence that CHECK/FK constraints are not enough:

Rule that is not enforced	Statement	Result
Loan with NO LoanObject lines is accepted	INSERT INTO Loan(... 'AGR-0002' ...) -- no lines inserted	ACCEPTED (loan 2 has 0 lines)
Same artifact on a second, overlapping open loan is accepted	INSERT INTO LoanObject(LoanID, ArtifactID, LineNo, LoanValue) VALUES (2,1,1,100)	ACCEPTED (artifact 1 is now on loans 1 and 2 at the same time)
Overlapping placement for the same artifact is accepted	INSERT INTO Placement(ArtifactID, ExhibitionID, SlotNo, FromDate) VALUES (1,1,2,'2026-02-01')	ACCEPTED (artifact 1 is in slot 1 and slot 2 in February)

Detection queries (executed) find the violations:

```
SELECT a.LoanID AS LoanA, b.LoanID AS LoanB, a.ArtifactID
FROM LoanObject a JOIN LoanObject b ON a.ArtifactID = b.ArtifactID AND a.LoanID < b.LoanID
JOIN Loan la ON la.LoanID = a.LoanID JOIN Loan lb ON lb.LoanID = b.LoanID
WHERE la.StartDate <= COALESCE(lb.ReturnDate, lb.DueDate) AND lb.StartDate <=
COALESCE(la.ReturnDate, la.DueDate)
```

Results of the detection queries (executed): loans with no lines (checked before the demonstration line was added) → LoanID 2, AGR-0002; artifacts on overlapping loans → artifact 1 on loans 1 and 2; overlapping placements → artifact 1, placements 1 and 2. These rules must be enforced with transactions, triggers or stored procedures (with locking), as also noted for ‘at least one line per loan’ and ‘authorised inspector’.

TASK 9 – Validate the Design with Use-Case Queries (3 Marks)

Execute eight queries (including overdue open loans and artifacts without condition evidence), display SQL and actual output including empty results, record counts, JOIN counts and median times for all four sizes, and demonstrate flat-file anomalies.

Answer:

9.1 Eight queries and their actual output (starter sample, run date 2026-11-03)

Q1 Open overdue loans

```
SELECT LoanID, AgreementNo, LenderPersonID, DueDate
FROM Loan
WHERE ReturnDate IS NULL AND DueDate < CURRENT_DATE
ORDER BY DueDate
```

LoanID	AgreementNo	LenderPersonID	DueDate
1	AGR-0001	1	2026-03-01

Q2 Artifacts without condition evidence

```
SELECT a.ArtifactID, a.AccessionNo, a.Title
FROM Artifact a
LEFT JOIN ConditionReport cr ON cr.ArtifactID = a.ArtifactID
WHERE cr.ReportID IS NULL
ORDER BY a.ArtifactID
```

Result: 0 rows (empty result).

After adding the supplementary rows (artifact MM-0003 with no loan; exhibition ‘Ancient Worlds’ with 2 slots and one placement of MM-0001):

ArtifactID	AccessionNo	Title
3	MM-0003	Silk Banner

Q3 Artifacts in each loan

```
SELECT l.LoanID, l.AgreementNo, a.ArtifactID, a.Title, lo.LineNo
FROM Loan l
JOIN LoanObject lo ON lo.LoanID = l.LoanID
JOIN Artifact a ON a.ArtifactID = lo.ArtifactID
ORDER BY l.LoanID, lo.LineNo
```

LoanID	AgreementNo	ArtifactID	Title	LineNo
1	AGR-0001	1	Bronze Bowl	1
1	AGR-0001	2	Ancient Pot	2

Q4 Custody period and duration

```
SELECT LoanID, StartDate, DueDate, ReturnDate,
  CAST(julianday(COALESCE(ReturnDate, CURRENT_DATE)) - julianday(StartDate) AS INTEGER) AS
CustodyDays
FROM Loan
ORDER BY StartDate
```

LoanID	StartDate	DueDate	ReturnDate	CustodyDays
1	2026-01-01	2026-03-01	NULL	306

Q5 Latest condition report per artifact

```
SELECT cr.ArtifactID, cr.ReportID, cr.InspectedAt, cr.ReportType, cr.Notes
FROM ConditionReport cr
JOIN (SELECT ArtifactID, MAX(InspectedAt) AS LatestAt
      FROM ConditionReport GROUP BY ArtifactID) x
  ON x.ArtifactID = cr.ArtifactID AND x.LatestAt = cr.InspectedAt
ORDER BY cr.ArtifactID
```

ArtifactID	ReportID	InspectedAt	ReportType	Notes
1	1	2026-01-01 10:00:00	DISPATCH	No visible damage
2	2	2026-01-01 10:15:00	DISPATCH	Minor surface wear

Q6 Loan lines and valuations

```
SELECT LoanID, ArtifactID, LineNo, LoanValue, DispatchCondition
FROM LoanObject
ORDER BY LoanID, LineNo
```

LoanID	ArtifactID	LineNo	LoanValue	DispatchCondition
1	1	1	24000	Good
1	2	2	17500	Minor surface wear

Q7 Exhibition slot placements

```
SELECT e.Name AS Exhibition, p.SlotNo, a.AccessionNo, a.Title,
       p.FromDate, p.ToDate
FROM Placement p
JOIN Exhibition e ON e.ExhibitionID = p.ExhibitionID
JOIN Artifact a ON a.ArtifactID = p.ArtifactID
ORDER BY e.ExhibitionID, p.SlotNo
```

Result: 0 rows (empty result).

After adding the supplementary rows (artifact MM-0003 with no loan; exhibition ‘Ancient Worlds’ with 2 slots and one placement of MM-0001):

Exhibition	SlotNo	AccessionNo	Title	FromDate	ToDate
Ancient Worlds	1	MM-0001	Bronze Bowl	2026-01-15	2026-03-01

Q8 Condition evidence count by artifact

```
SELECT a.ArtifactID, a.AccessionNo, a.Title,
       COUNT(cr.ReportID) AS ReportCount
FROM Artifact a
LEFT JOIN ConditionReport cr ON cr.ArtifactID = a.ArtifactID
GROUP BY a.ArtifactID, a.AccessionNo, a.Title
ORDER BY ReportCount, a.ArtifactID
```

ArtifactID	AccessionNo	Title	ReportCount
1	MM-0001	Bronze Bowl	1
2	MM-0002	Ancient Pot	1

After adding the supplementary rows (artifact MM-0003 with no loan; exhibition ‘Ancient Worlds’ with 2 slots and one placement of MM-0001):

ArtifactID	AccessionNo	Title	ReportCount
3	MM-0003	Silk Banner	0
1	MM-0001	Bronze Bowl	1
2	MM-0002	Ancient Pot	1

Q1 returns loan 1 because it has no ReturnDate and its DueDate (2026-03-01) is before the run date. Q2 and Q7 are legitimately empty on the starter data – every artifact has a report and nothing is placed – and are shown as ‘0 rows’ rather than omitted; after the supplementary rows Q2 lists MM-0003 and Q8 ranks it first with 0 reports.

9.2 Result counts, JOIN counts and median times for the four populations

Each query was executed five times after loading and `ANALYZE`; the table gives the **median** elapsed milliseconds (client-side, full result fetched) on the same machine, same indexes and no restart between sizes. JOIN counts are the explicit JOIN keywords in the SQL text.

Query	JOINs	rows @100	rows @1,000	rows @10,000	rows @100,000
Q1 Open overdue loans	0	4	22	254	2625
Q2 Artifacts without condition evidence	1	730	7306	73028	730247
Q3 Artifacts in each loan	2	20	200	2000	20000
Q4 Custody period and duration	0	10	100	1000	10000
Q5 Latest condition report per artifact	1	20	194	1972	19753
Q6 Loan lines and valuations	0	20	200	2000	20000
Q7 Exhibition slot placements	2	10	98	982	9988
Q8 Condition evidence count by artifact	1	750	7500	75000	750000

Query (median ms of 5 runs)	100 lenders	1,000 lenders	10,000 lenders	100,000 lenders
Q1 Open overdue loans	0.01	0.02	0.19	2.15
Q2 Artifacts without condition evidence	0.32	4.13	43.08	527.93
Q3 Artifacts in each loan	0.02	0.2	2.88	39.46
Q4 Custody period and duration	0.01	0.07	0.81	8.87
Q5 Latest condition report per artifact	0.03	0.27	2.96	38.19
Q6 Loan lines and valuations	0.01	0.1	1.07	11.57
Q7 Exhibition slot placements	0.01	0.09	1.61	24.7
Q8 Condition evidence count by artifact	0.75	10.5	95.37	1054.88

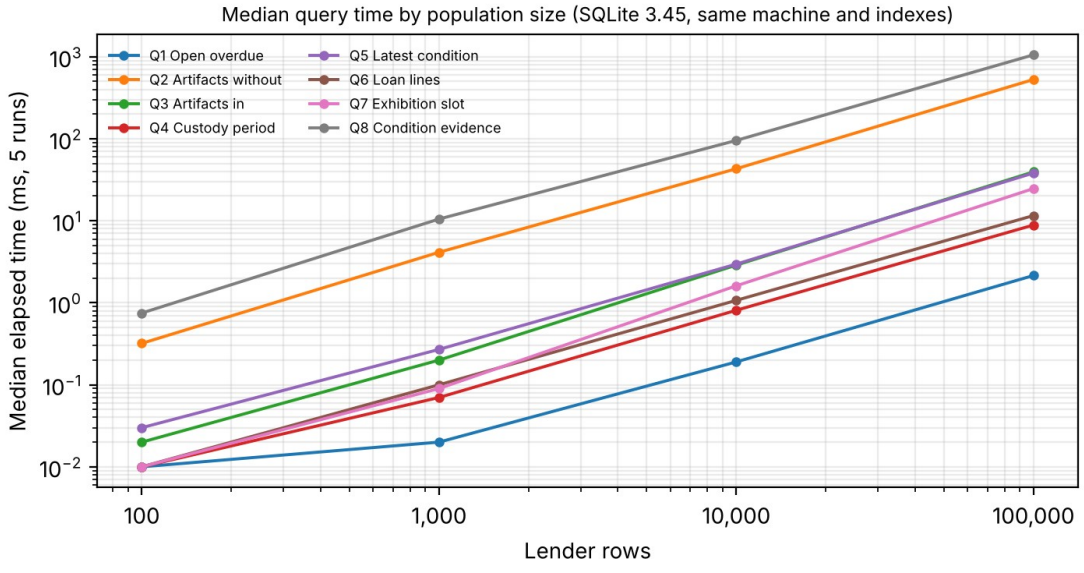


Figure 7 – Median elapsed time versus lender population (log–log)

Reading the results. Q2 and Q8 return or group every artifact (750,000 at the top size), so they grow almost linearly with the artifact count (Q2: 44.5 ms → 515 ms, Q8: 92 ms → 1,095 ms for ten times more data). Q1 is answered through the index on (DueDate, ReturnDate) (`SEARCH Loan USING INDEX ix_loan_due`) and stays at about 2 ms at 100,000 lenders; Q2 uses the covering index on ConditionReport(ArtifactID, InspectedAt) for its LEFT JOIN (`SEARCH cr USING COVERING INDEX ix_cr_artifact`). At 100 lenders the times (0.01–0.9 ms) are dominated by call overhead and should not be compared with each other. Times are SQLite times on this machine and are not MySQL measurements.

9.3 Flat-file anomalies (executed)

A flat file FlatLoan(LoanID, ArtifactID, AgreementNo, LenderName, LenderPhone, ArtifactTitle, DueDate, ConditionNotes) with key (LoanID, ArtifactID) and three rows (two lines of loan 1 for Asha Raman, one line of loan 2 for Bala Kumar) was created.

Anomaly	Operation	Actual outcome
Insertion	Register lender ‘Charu Devi’ with no loan (LoanID/ArtifactID NULL)	NOT NULL constraint failed: FlatLoan.LoanID – the lender cannot be stored without inventing a loan row
Update	Change Asha's phone on one row only (LoanID 1, ArtifactID 1)	Her two rows now hold 2 different phone numbers: 95555, 90001
Deletion	Delete the only line of loan 2	Rows left for Bala Kumar: 0 – the loan, the lender and the phone are lost together

The normalised design (PERSON/LENDER_ROLE, LOAN, ARTIFACT, LOAN_OBJECT, CONDITION_REPORT) prevents all three: a lender exists without a loan, a phone number is stored once, and deleting a line leaves the loan and lender intact.

TASK 10 – Compare with File-Based, Network and Hierarchical Models(4 Marks)

Compare the selected design with file processing, a hierarchical tree and a network model; trace an exhibition inventory through placements.

Answer:

10.1 File-based model

Separate lender, loan, artifact, exhibition and condition-report files must be matched by key in application code and duplicated descriptions kept in step by hand. A due date copied into a gallery file creates update inconsistency; finding artifacts with no condition report means comparing two files and handling missing matches manually. It is simple for small lists but weak for concurrent updates, shared identity and historical integrity – the flat-file anomalies of Task 9 are the measured example.

10.2 Hierarchical model

```
LENDER                          EXHIBITION
 └── LOAN                        └── DISPLAY_SLOT
      └── LOAN_OBJECT                 └── PLACEMENT
           └── ARTIFACT                    └── ARTIFACT (reference/copy needed)
                └── CONDITION_REPORT
```

A tree gives each child one parent path. Navigation lender → loan → artifacts is efficient, but an artifact has successive loans and reports and is placed in different exhibitions over time, so it must be duplicated or reached by pointer-like links. A person who is both lender and curator would appear in two trees.

10.3 Network model

```
LENDER   --owner/member set--> LOAN
LOAN     --owner/member set--> LOAN_OBJECT
ARTIFACT --owner/member set--> LOAN_OBJECT
ARTIFACT --owner/member set--> CONDITION_REPORT
EXHIBITION --owner/member set--> DISPLAY_SLOT
```

Several owner–member paths allow an artifact to be reached through every loan occurrence and its reports, which is more flexible than a tree, but the program must know which set to follow and navigate record by record.

10.4 Two case queries traced through each model

Query	File-based	Hierarchical	Network	Relational DBMS (executed)
Open overdue loans (Q1)	Scan the loan file and filter dates	Traverse each lender's loan list	Follow the lender → loan set and test dates	`WHERE ReturnDate IS NULL AND DueDate < CURRENT_DATE` – 1 row on the sample, 2,625 rows at 100,000 lenders in about 2 ms
Artifacts without evidence (Q2)	Compare artifact and report files	Visit every artifact branch and test for a missing child	Follow artifact → report set and find empty ones	`LEFT JOIN … WHERE cr.ReportID IS NULL` – 0 rows on the sample, 730,247 rows at the top size

10.5 Three-schema architecture and concurrency

External schema (operational views): a Registrar view of open loans and returns, a Curator view of placements, a Conservator view of condition history – each shows only what the role needs. **Conceptual schema:** the shared logical model (PERSON … PLACEMENT) with its constraints, as executed in Task 8. **Internal schema:** pages, the indexes created in Task 7/8 (for example `ix_loan_due`, `ix_cr_artifact`), clustered and secondary access paths, record formats and logs. When two users update the same loan, the **transaction manager** groups the changes, the **lock/concurrency-control manager** orders conflicting access, the **buffer manager** serves cached pages, the **log/recovery manager** records changes for rollback and crash recovery, and the **integrity-constraint manager** checks keys, foreign keys and CHECKs. Isolation prevents a half-finished update being seen, and loan dates and related custody evidence should change in one atomic transaction.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

Recommend one alternative using the schema, query outputs and constraint tests; defend keeping custody evidence distinct from ownership history.

Answer:

Recommendation: Alternative B combined with C – an independently identified LOAN, LOAN_OBJECT as the loan–artifact associative entity, PERSON as the shared identity with lender/curator/registrar roles, and separate PLACEMENT and CONDITION_REPORT paths so that where an object is displayed is never confused with what condition was observed.

11.1 Requirement-to-model traceability

Requirement	Model element	How it is satisfied / evidence
One lender per loan	LOAN.LenderPersonID FK	NOT NULL FK to LENDER_ROLE; a loan naming a person who is not a registrar was rejected
Several artifacts per loan	LOAN_OBJECT	One row per line; Q3 listed both artifacts of AGR-0001
Successive loans of an artifact	LOAN_OBJECT history	Same ArtifactID under different LoanIDs; no overwrite
Condition evidence by physical object	CONDITION_REPORT	Composite FK (LoanID, ArtifactID); report for an artifact not on the loan rejected; Q2/Q5/Q8
Slots numbered within an exhibition	DISPLAY_SLOT composite PK	Placement into a missing slot rejected; Q7
Curator may also lend	PERSON + LENDER_ROLE + EMPLOYEE	One PersonID with two role rows
Ownership differs from custody	Ownership history (Task 12)	LOAN rows unchanged when shares change

11.2 Evidence versus untested assertions

Tested by execution: key uniqueness, 16 invalid writes (all rejected), referential integrity at four sizes (0 violations), the eight queries and their counts, the three cross-row gaps and flat-file anomalies. **Not tested / assertions only:** behaviour on a real MySQL server, concurrency under simultaneous sessions, the proposed triggers, and the business assumptions that AccessionNo and AgreementNo are unique by policy.

11.3 Remaining limitations and growth to 100,000 lenders

Cross-table rules still need transactions or triggers: at least one line per confirmed loan, no overlapping loans or placements for one artifact (all three were accepted by the schema in Task 8), authorised inspector, and ownership shares totalling 100 %. Identity assumptions (one person = one PersonID, unique accession numbers) depend on registration discipline. At 100,000 lenders the generated database holds 750,000 artifacts, 10,000 loans and 36,490 reports; the indexed queries stayed in the low milliseconds (Q1 2.2 ms, Q3 42 ms, Q5 39 ms) while the queries that must touch every artifact (Q2 515 ms, Q8 1,095 ms) grew roughly linearly, so those should be paginated, restricted by collection unit or served from a maintained summary. Indexes speed lookups but cost storage and write time. The design is chosen because it preserves the real-world distinctions and historical evidence, not because of the number of tables.

TASK 12 – Re-evaluate and Adapt the Design (Changed Requirement) (8 Marks)

Artifacts may have shared legal ownership and ownership shares can change while an existing loan remains in force; custody must stay distinct from ownership history. Decide RETAIN, MODIFY or REDESIGN.

Answer:

A. Impact analysis (2 marks)

Invalidated assumption: an artifact has one current lender who can be treated as its sole owner. A lender is the party responsible for a loan/custody agreement; a legal owner holds an ownership share. They may coincide but are not equivalent: one lender may coordinate a loan for several co-owners, and a co-owner need not be the lender. **Affected structures:** ARTIFACT, PERSON/ORGANIZATION, LENDER_ROLE, LOAN and the (missing) ownership association; the business key ‘current lender’ cannot express several owners, percentage shares, effective dates or transfers, so LenderPersonID alone no longer describes the fact ‘who owns this artifact on date D’. LOAN.LenderPersonID stays valid for custody responsibility. **Across the schema levels:** the operational view shows the responsible lender and, separately, owners and shares effective on the relevant date; the conceptual schema gains ARTIFACT_OWNER and OWNERSHIP_PERIOD; the internal schema gains an index on (ArtifactID, ValidFrom, ValidTo). Existing loan dates and condition reports do not change.

B. Evaluation of the original design (2 marks)

EXHIBITION and DISPLAY_SLOT still describe where an artifact is shown and are not affected; placement cannot be used to infer ownership. Stable primary keys (ArtifactID, PersonID, LoanID, ReportID, ExhibitionID) remain valid. What must change is the assumption that one person is the only legal owner; new temporal constraints are needed: shares valid over date intervals, no conflicting intervals for the same owner and artifact, and total share = 100 % at every effective date. **Rejected alternatives revisited:** A (dated lender–artifact relationship) still cannot represent shared ownership because custody is not ownership; B remains the strongest custody model but needs a separate ownership association; C remains useful for shared PERSON identity but needs an ORGANIZATION owner possibility.

C. Adaptation – decision: MODIFY (2 marks)

Retain LOAN and LOAN_OBJECT (custody) and **add** a temporal many-to-many ownership structure – the smallest change that preserves all historical custody evidence. ARTIFACT_OWNER is a union category of PERSON or ORGANIZATION; OWNERSHIP_PERIOD is the associative/history entity holding SharePercent, ValidFrom, ValidTo. The old single-owner/current-lender association is not reused for ownership; it is kept for custody only.

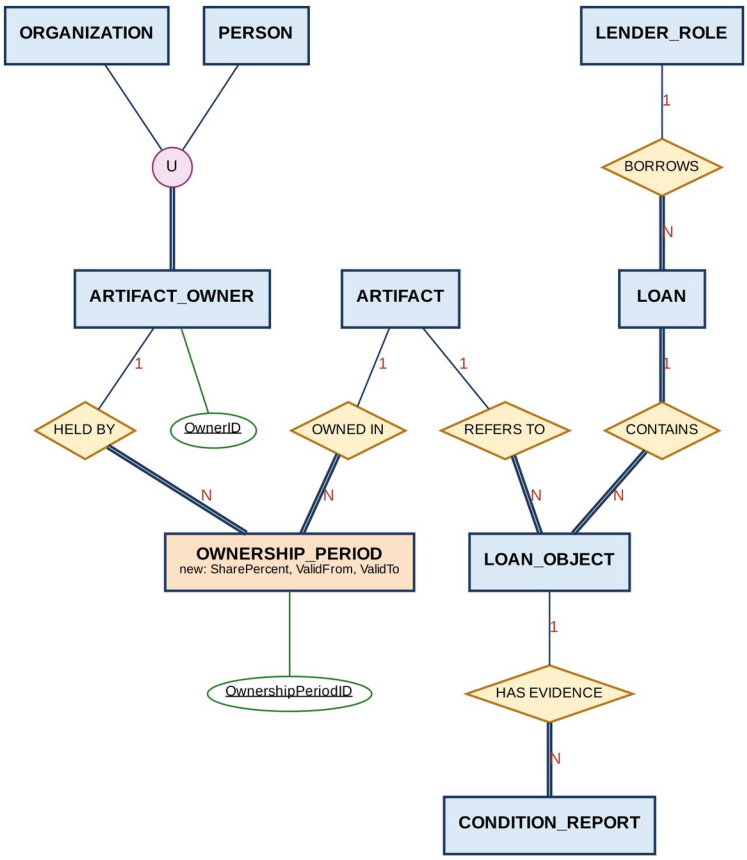


Figure 8 – Revised EER: ARTIFACT_OWNER (union) and OWNERSHIP_PERIOD added; custody path unchanged

```
ORGANIZATION(OrganizationID PK, LegalName, RegistrationNo UNIQUE)
ARTIFACT_OWNER(OwnerID PK, OwnerType, PersonID FK NULL, OrganizationID FK NULL,
               CHECK exactly one of PersonID / OrganizationID is non-NULL)
OWNERSHIP_PERIOD(OwnershipPeriodID PK, ArtifactID FK, OwnerID FK, SharePercent DECIMAL(5,2),
               ValidFrom DATE, ValidTo DATE NULL, UNIQUE(ArtifactID, OwnerID, ValidFrom),
               CHECK (SharePercent > 0 AND SharePercent <= 100),
               CHECK (ValidTo IS NULL OR ValidTo >= ValidFrom))
```

Migration: (1) create the owner identities and the two new tables; (2) convert an existing single owner/current lender into a 100 % ownership row only if the museum confirms the old field really meant legal ownership – if it meant custody only, obtain verified ownership data instead; (3) keep every existing ARTIFACT, LOAN, LOAN_OBJECT, CONDITION_REPORT and PLACEMENT key and timestamp and never rewrite historical loans; (4) validate shares and dates per artifact and record the migration batch; (5) switch operational views to read the lender from LOAN and owners from the ownership tables.

D. Evidence-based defence (2 marks)

The revised structure was **executed** (the executed table is named `OwnershipPeriod`; the draft query used `Ownership_Period`). Test data: artifact 1 owned 60 % Asha / 40 % Bala until 2026-06-30, then 50 % Asha / 30 % Bala / 20 % Heritage Trust of Madurai – the share change happens while loan 1 (started 2026-01-01) is still open; artifact 2 is 100 % Trust; artifact 3 deliberately has only 70 % (a faulty migration).

```
SELECT l.LoanID, l.AgreementNo, a.AccessionNo, l.StartDate,
       COALESCE(p.GivenName || ' ' || p.FamilyName, org.LegalName) AS Owner, op.SharePercent,
op.ValidFrom, op.ValidTo
FROM Loan l JOIN LoanObject lo ON lo.LoanID = l.LoanID JOIN Artifact a ON a.ArtifactID =
lo.ArtifactID
JOIN OwnershipPeriod op ON op.ArtifactID = a.ArtifactID
JOIN ArtifactOwner ao ON ao.OwnerID = op.OwnerID
LEFT JOIN Person p ON p.PersonID = ao.PersonID LEFT JOIN Organization org ON org.OrganizationID =
ao.OrganizationID
WHERE a.ArtifactID = 1 ORDER BY op.ValidFrom, Owner
```

LoanID	AgreementNo	AccessionNo	StartDate	Owner	SharePercent	ValidFrom	ValidTo
1	AGR-0001	MM-0001	2026-01-01	Asha Raman	60	2025-01-01	2026-06-30
1	AGR-0001	MM-0001	2026-01-01	Bala Kumar	40	2025-01-01	2026-06-30
1	AGR-0001	MM-0001	2026-01-01	Asha Raman	50	2026-07-01	NULL
1	AGR-0001	MM-0001	2026-01-01	Bala Kumar	30	2026-07-01	NULL
1	AGR-0001	MM-0001	2026-01-01	Heritage Trust of Madurai	20	2026-07-01	NULL

The loan row is unchanged (LoanID 1, StartDate 2026-01-01, DueDate 2026-03-01) although ownership changed, so custody evidence is not rewritten. **Query that reveals an incorrect migration** (total active share ≠ 100 %):

```
SELECT a.ArtifactID, COALESCE(SUM(op.SharePercent), 0) AS TotalActiveShare
FROM Artifact a
LEFT JOIN OwnershipPeriod op ON op.ArtifactID = a.ArtifactID AND op.ValidFrom <= '2026-11-01' AND
(op.ValidTo IS NULL OR op.ValidTo >= '2026-11-01')
GROUP BY a.ArtifactID
HAVING COALESCE(SUM(op.SharePercent), 0) <> 100.00
```

Actual output for 2026-11-01: (ArtifactID, TotalActiveShare) = (3, 70) – artifact 3 is flagged; artifacts 1 and 2 total 100 %. The same query for 2026-03-01 also returns (3, 70). The detached-evidence query (condition reports with no matching loan line) returned 0 rows.

Invalid ownership write	Actual SQLite message
Owner row with BOTH person and organisation (CHECK)	CHECK constraint failed: (PersonID IS NOT NULL AND OrganizationID IS NULL AND OwnerType='PERSON') OR (PersonID IS NULL AND OrganizationID IS NOT NULL AND OwnerType='ORGANIZATION')
Owner row with NEITHER (CHECK)	CHECK constraint failed: (PersonID IS NOT NULL AND OrganizationID IS NULL AND OwnerType='PERSON') OR (PersonID IS NULL AND OrganizationID IS NOT NULL AND OwnerType='ORGANIZATION')
Share of 120 percent (CHECK)	CHECK constraint failed: SharePercent > 0 AND SharePercent <= 100
ValidTo before ValidFrom (CHECK)	CHECK constraint failed: ValidTo IS NULL OR ValidTo >= ValidFrom
Duplicate (artifact, owner, ValidFrom) (UNIQUE)	UNIQUE constraint failed: OwnershipPeriod.ArtifactID, OwnershipPeriod.OwnerID, OwnershipPeriod.ValidFrom

Effect on generated data: with about two owners per artifact the ownership association holds roughly twice as many rows as artifacts (about 1.5 million rows at 750,000 artifacts) plus history rows for share changes; lender, loan, loan-line and condition-report counts do not change. **Existing views keep their interface:** they continue to return LoanID, Lender, Artifact, StartDate, DueDate and ReturnDate; an optional owner-summary view joins the ownership tables when legal title is needed. **Cost:** more association rows, an extra index and temporal validation (sum of shares = 100 %, no overlapping intervals – which, like the loan overlaps in Task 8, are cross-row rules needing a trigger or procedure, not CHECKs). **Decision:** MODIFY, because only a separate, time-aware ownership association can represent shared ownership and changing shares without confusing legal title with temporary custody.

Mark distribution

Task	Marks	Task	Marks
1 – Requirements and alternatives	5	7 – Relational mapping	4
2 – Entities, attributes, keys	2	8 – Implementation and instances	4
3 – Relationships and constraints	6	9 – Design validation	3
4 – ER diagram	4	10 – Data model comparison	4
5 – EER modelling	3	11 – Final recommendation	4
6 – Weak entities, complex attributes	3	12 – Re-evaluation challenge	8

Total: 50 (Tasks 1–11 = 42, Task 12 = 8).

Signature of the student:

APPENDIX A – Runner used for Tasks 8, 9 and 12 (run_meridian.py, extract)

The runner creates the schema, loads the starter sample, executes the invalid writes and the eight queries, then generates and measures the four populations with `random.Random(4211)`. The data-generation function is reproduced below (SEED = 4211, `newdb()` creates the schema and indexes); the remaining parts are the SQL already shown in this report. To repeat the run on MySQL, replace `sqlite3` by a MySQL connector, use `ddl_mysql.sql` for the schema and keep the same generation rules.

```
def generate(db, N):
    rng = random.Random(SEED)
    c = newdb(db); c.execute("PRAGMA synchronous=OFF"); c.execute("PRAGMA journal_mode=MEMORY")
    n_art = int(N * 7.5); n_loan = N // 10; n_cur = max(3, round(N * 35 / 1200)); n_reg = max(2, round(N /
100)); n_exh = 12; slots = 20
    c.executemany("INSERT INTO CollectionUnit(UnitName) VALUES (?)", [(u,) for u in ["Ancient
History","Decorative Arts","Textiles","Sculpture","Paintings","Manuscripts","Archaeology","Natural History"]])
    c.executemany("INSERT INTO CollectionType(TypeName, Description) VALUES (?,?)", [("CERAMIC","Ceramic
artifact"),("METAL","Metal artifact"),("TEXTILE","Textile artifact"),("PAPER","Paper-based artifact")])
    people = []; pid = 0
    def mk(n, tag):
        nonlocal pid
        out = []
        for i in range(n):
            pid += 1; out.append(pid)
            people.append((pid, rng.choice(FIRST), rng.choice(LAST), f"{tag}{pid}@example.test"))
        return out
    lenders = mk(N, "l"); curators = mk(n_cur, "c"); registrars = mk(n_reg, "r")
    c.executemany("INSERT INTO Person(PersonID, GivenName, FamilyName, Email) VALUES (?,?,?,?)", people)
    c.executemany("INSERT INTO PersonPhone VALUES (?,?,?)", [(p, f"9{rng.randrange(10**9):09d}", "MOBILE") for
p in lenders])
    c.executemany("INSERT INTO LenderRole(PersonID, LenderSince) VALUES (?,?)", [(p,
str(datetime.date(2020,1,1) + datetime.timedelta(days=rng.randrange(2000)))) for p in lenders])
    c.executemany("INSERT INTO Employee VALUES (?,?,?,?)", [(p, f"EMP-{p:06d}", "2022-01-01", "CURATOR") for p
in curators] + [(p, f"EMP-{p:06d}", "2022-01-01", "REGISTRAR") for p in registrars])
    c.executemany("INSERT INTO Curator VALUES (?,?)", [(p, "General") for p in curators])
    c.executemany("INSERT INTO Registrar VALUES (?,?)", [(p, "STANDARD") for p in registrars])
    c.executemany("INSERT INTO Artifact(ArtifactID, AccessionNo, Title, CollectionUnitID, CollectionTypeID,
ArtifactValue) VALUES (?,?,?,?,?,?)",
        [(i, f"MM-{i:07d}", rng.choice(TITLES), rng.randint(1, 8), rng.randint(1, 4), round(rng.uniform(500,
90000), 2)) for i in range(1, n_art + 1)])
    c.executemany("INSERT INTO Exhibition(Name, StartDate, EndDate) VALUES (?,?,?)", [(f"Exhibition {i}",
"2026-01-01", "2026-12-31") for i in range(1, n_exh + 1)])
    c.executemany("INSERT INTO DisplaySlot VALUES (?,?,?)", [(e, s, f"Gallery {e} - Case {s}") for e in
range(1, n_exh + 1) for s in range(1, slots + 1)])
    base = datetime.date(2025, 1, 1); loans = []; lines = []; reps = []; places = []
    for L in range(1, n_loan + 1):
        start = base + datetime.timedelta(days=rng.randrange(0, 670))          # 2025-01-01 .. 2026-10-31
        due = start + datetime.timedelta(days=rng.randint(30, 180))
        returned = rng.random() < 0.7
        ret = (due + datetime.timedelta(days=rng.randint(-10, 10))) if returned else None
        if ret and ret < start: ret = start
        loans.append((L, rng.choice(lenders), f"AGR-{L:07d}", str(start), str(due), str(ret) if ret else None,
rng.choice(registrars)))
        for ln, a in enumerate(rng.sample(range(1, n_art + 1), 2), 1):          # parent keys drawn from
existing artifacts
            lines.append((L, a, ln, round(rng.uniform(400, 80000), 2), rng.choice(["Good", "Minor surface
wear", "Stable"])))
            reps.append((L, a, rng.choice(curators), f"{start} 10:00:00", "DISPATCH", "Dispatch inspection",
None))
            if ret and rng.random() < 0.9: reps.append((L, a, rng.choice(curators), f"{ret} 15:00:00",
"RETURN", "Return inspection", None))
            if rng.random() < 0.2: reps.append((L, a, rng.choice(curators), f"{start +
datetime.timedelta(days=15)} 11:00:00", "IN_DISPLAY", "Display check", None))
            if rng.random() < 0.5: places.append((a, rng.randint(1, n_exh), rng.randint(1, slots), str(start +
datetime.timedelta(days=5)), str(due)))
    c.executemany("INSERT INTO Loan(LoanID, LenderPersonID, AgreementNo, StartDate, DueDate, ReturnDate,
RegistrarPersonID) VALUES (?,?,?,?,?,?,?)", loans)
    c.executemany("INSERT INTO LoanObject VALUES (?,?,?,?,?)", lines)
    c.executemany("INSERT INTO ConditionReport(LoanID, ArtifactID, InspectorPersonID, InspectedAt, ReportType,
Notes, EvidenceURI) VALUES (?,?,?,?,?,?,?)", reps)
    c.executemany("INSERT INTO Placement(ArtifactID, ExhibitionID, SlotNo, FromDate, ToDate) VALUES
(?,?,?,?,?)", places)
    c.commit()
    return c
```